Lecture 12: Recap

Lui Holder-Pearson

October 15, 2026

Learning Goals

- Understand the entire ENCE260 computer architecture content.

Further Reading

ENCE260 Learn page - Computer Architecture section

References

Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapters 13 & 14.

L1: Intro

- The man at the front of the room is Lui, who will be lecturing the computer architecture aspect of ENCE260.
- There are 11 lectures, 6 tutorials, 4 quizzes, and a portion of the exam throughout the semester.
- The three main components of a computer are:
  1. Memory and Buses
  2. Central processing unit
  3. Internal peripherals
- Understand logic statements, variables, and values:
  → Logic statements are sentences that answer a yes/no question.
  → Logic variables are the programming variable names in which we store the answers.
  → Logic values are the values stored within the above variables: either 0/1, `FALSE`/`TRUE`, or the physical representation of how this electrically achieved.

L2: Data Representation

- Understand the need to store numerical values in computers.
  → The everyday world contains a large amount of continuous numerical information, and, given computers interact directly with the real world, computers must be able to handle numerical information.

- Understand how groups of logical variables are used to encode numerical information — binary.
  → As each digital variable can contain a single piece of digital information, the information we encode within each bit of a sequence is "does the sum total of the number include 2^n?", where n is the position of the digital variable/memory bit within the sequence:
  → $Value = \sum d_n \times 2^n$

- Understand how the representations of negative hexadecimal numbers correlate to digital information.
  → Negative numbers are typically stored as two's complement, where a `1` in the nth position shows a negative, and the value of a negative number is equal to:
  $Magnitidue = \sum_{n-1}(\overline{d_n} \times 2^n) + 1$
  → Hexadecimal is the base-16 number system, and is extremely useful because each digit is able to be directly, independently translated into its binary representation.
  → There is a method of storing non-integer numerical information in scientific notation.

- Understand the need to know which sequence is being used, and how this pertains to C programming.
  → We know that the interpretation of an 8-bit sequence can be vastly different depending on the interpretation: numbers can be signed integers, unsigned integers, or floats. Furthermore, 8-bit sequences can encode characters, or pointers.
  → In C, we declare the data type of a variable before we first use it, so the computer can be properly instructed how to handle it.

L3: Memory and buses

- Understand rudimentary logic functions and logic gates:
  → The basic functions `AND`, `ORNOT`, and `XOR` form the basic logic gates, allowing more complex logic circuits.

- Understand how sequential logic components are used to implement computer memory.
  → Sequential logic enables memory through the use of circuits like the SR latch, and the D flip flop, where the output while stay the same except in specific circumstances, thus enacting memory.

- Understand the arrangement of memory within embedded systems.
  → Memory is formed of arrays of flip flops to form registers.
  → There is also memory for storing data variables in RAM.
  → Every individual byte is has its own address, which is a number specifying its location.

- Know what pointers within C are, and how they pertain to memory arrangement.
  → Pointers are a special variable type which store the address of the variable.
  → Pointers are used in C through the use of *, meaning 'the variable at this address' (e.g. `*ptr_to_charB`), and &, which means 'the address of this variable'.

- Briefly understand now information is transmitted throughout an embedded computer.
  → The data bus is a bidirectional bus with width equal to the word size for transferring information.
  → The address bus is used to specify the address which is being read/written.
  → The control bus governs the data bus, in terms of directionality and synchronisation.

L4: Boolean/Digital Logic

- Understand logic statements, variables, and values:
  → Logic statements are sentences that answer a yes/no question.
  → Logic variables are the programming variable names in which we store the answers.
  → Logic values are the values stored within the above variables: either 0/1, `FALSE`/`TRUE`, or the physical representation of how this electrically achieved.

- Understand logic functions.
  → Logic functions are functions which have logical inputs an outputs, just as numerical functions take numerical inputs and outputs.
  → The *fundamental* logic functions are `NOT`, `OR`, and `AND`.

- Understand and use tools to simplify logic functions.
  → There are several algebraic tools that we can use to reduce logic equations to more simplistic ones.
  → De Morgan's laws specifically are useful because they allow reduction to a single operation.

L5: CPU

- Understand the subcomponents of a CPU, and what each one does.
  → **ALU**: actually performs the operations required, and in doing so generates the data that can be used in subsequent operations.
  → **General purpose registers**: Physically close to the ALU, provides memory for the inputs to, and outputs from the ALU.
  → **Control/execute unit**: Responsible for controlling the data flow to and from the general purpose registers, and acts as a finite state machine in decoding the instructions from the program memory.
  → **Control registers**: Stores metadata about the latest calculations.

- Have an understanding of how the adder of an ALU is formed with logic gates.
  → **Half adders** are used to compare two individual bits, with outputs `sum`, and `carry`.
  → **Full adders** use two half adders cascaded. Full adders themselves are cascaded to form a complete n-bit addition unit.

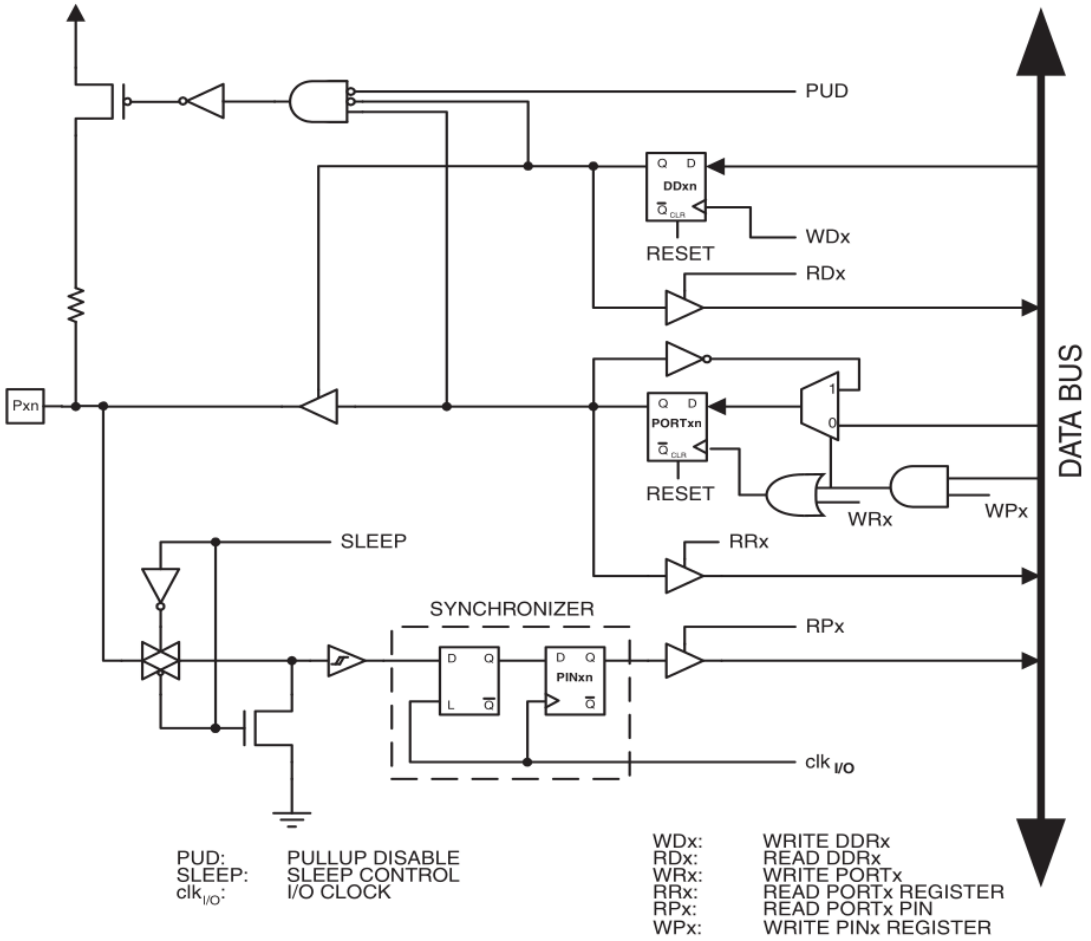
- Understand the logic required to implement a one-bit comparator.
  → $gt = A$ `AND` $\overline{B}$
  → $eq = (A$ `AND` $B)$ `OR` $(\overline{A}$ `AND` $\overline{B})$
  → $lt = \overline{A}$ `AND` B

L6: Combinational logic circuits

- Understand the use within embedded systems, and design of:
  - Decoders and encoders;
    → Decoders decode a n-bit binary number into exactly one true signal of 2^n possible outputs. They are used for selecting the byte in memory being read from, for example.
    → Encoders are the inverse, where exactly one of 2^n inputs are true, the corresponding number of the input is encoded as a binary number within a n-bit output.
  - Multiplexers and demultiplexers; and
    → Multiplexers are devices which map one of several inputs to the output. The most common configuration is Many-to-one, where n selection pins chose one of 2^n inputs to be connected to one output. Bus multiplexers also exist, where a collection of inputs is mapped to multi-bit output.
    → Muxes are used to select which specific operation is selected as the output from an ALU, for example.
    → A demultiplexer is the inverse: there is one single input (or multi-bit for bus demuxers) that is routed to one of several outputs. For example, to choose which peripheral device to communicate with.
  - Subtraction within an ALU.
    → Subtraction is implemented by converting the subtrahend into two's complement using an XOR gate on each bit of the subtrahend to invert it, and using the C_{in} property of the addition unit to add `1`.
- Understand how we use bus-width combinational logic circuits to implement the control within computers.
  → We have seen how decoders and muxes work to select the memory byte, and muxes to select the output of the ALU. We will see how this interfaces with software, and more specifics in Lecture 11 - Program Execution.

L7: GPIO

Figure 12-2. General Digital I/O(1)



L8: Sequential logic circuits

- Understand the design of:
  - Shift registers;
    → Shift registers are when the output of one flip-flop is connected to the input of the next, with a common clock. Used for serial-parallel conversions.
  - Counters; and
    → Counters use the inverted output of the flip-flop as the input. This therefore acts as a clock divider, or prescaler. Multiple connected flip-flops form a ripple counter.
    → Synchronous counters use additional circuitry to predict the 'next' state of the system as a function of the current state.
  - Finite state machines.
    → Finite state machines are control systems used to specify the state of a system.
- Understand the role of the above within embedded systems.
  → Each of them are contained in some format within a microcontroller to achieve base, or additional functionality.

L9: Peripherals

- Understand the need and the use of peripherals within microcontrollers.
  → Internal peripherals allow specific functionality to be implemented in hardware, rather than with expensive, extensive software.

- Understand the use of timer and counter peripherals with the ATMega32u2.
  → Timers and counters within the ATMega32u2 are available. There are two timers, one 8-bit, and one 16-bit.
  → Each have compare and PWM capability, and the 16-bit has capture capability.

- Understand how analogue signals are estimated with the ATMega32u2:
  → Output: through PWM, where the total portion of time a digital square wave is one is a function of register configuration, and thus electronically configurable.
  → Input: through use of an on-board analogue comparator, the time taken to charge a capacitor is measured, and compared to the time take to charge to a known voltage.
  → Subsequently, $V_{in} = V_{Ref} \cdot \frac{t_{in}}{t_{Ref}}$.

L10: Execution

- Understand the *fetch*, *decode*, and *execute* states.
  → These are the three states of the control and execute FSM, where the specific state transitions are governed by a clock and the overall programme flow.
  → Fetch retrieves an instruction from the programme memory,
  → Decode informs the control and execute unit how to configure the rest of the CPU, and the address and control buses.
  → Execute is when the instruction is carried out, and the result stored in the appropriate location.

- Understand how *instructions* are stored within the ATMega32u2.
  → Instructions are stored within 16-bit opcode instructions. Each opcode contains the specific sequence of `1`s and `0`s to specify the operation, as well as additional function parameters, such as memory/register addresses, and/or constants.

- Be able to interpret basic AVR instructions.
  → Similar to the examples we saw: `ldd`, `adiw`, `breq`.

- Understand how the program counter is used and manipulated for programme flow.
  → The program counter (PC) is a very special pointer which contains the location in memory of the next instruction to be read.
  → It is manipulated with branching `brXX` instructions, otherwise it is typically incremented by one every instruction.

- Be able to interpret some basic examples of assembly language.

L11: Function calls

- Loosely understand the memory layout of the ATMega32u2
  → The ATMega32u2 is memory mapped, with the general registers, the I/O registers, extended I/O registers, and finally the SRAM being mapped into a single addressing system.
- Understand what is meant by the term program *context*.
  → The program context is all the required information to complete the current function.
- Understand what the *stack* is and how it is manipulated:
  → The stack is the section of RAM with the highest addresses. It is used entirely in a last-in, first-out manner: Only the data most recently written to the stack is accessible.

  Specifically:

  - Understand what `push`, `pop`, `(r)call`, and `ret` mean and do.
    → `push Rr` stores the value of register r on the stack.
    → `pop Rd` retrieves the current value at the top of the stack into register Rd.
    → `call k` calls a subroutine at location k in the program memory.
    → `ret` returns from the current subroutine, retrieving the address of the next instruction from the stack.
  - Understand what the stack pointer is, and how it is used and manipulated.
    → The stack pointer is the pointer to the current address of the first free space at the top of the stack. It is manipulated implicitly through the use of the `push`, `pop`, `call`, and `ret` operations.
  - Understand what a stack frame is, and how it pertains to function calls.
    → A stack frame stores the program context. Specifically, it will store:
    * The value of the program counter on return from the function.
    * Register contents before the function call
    * Function arguments.
    * Function local variables.
    * Variables returned by the function.